

Projet Semestriel 1 ING (cours aménagés)

Développement d'une application web de gestion collaborative de budget personnel

1. Contexte du projet

Dans la vie quotidienne, la gestion du budget personnel est essentielle, aussi bien pour un individu que pour un groupe partageant des dépenses, comme une famille, des colocataires ou une équipe de projet. Les outils existants sont parfois trop complexes, peu collaboratifs ou non adaptés.

Ce projet consiste à concevoir et développer une **application web de gestion collaborative de budget personnel** permettant à plusieurs utilisateurs de suivre leurs revenus, dépenses, objectifs financiers et budgets partagés, à travers une interface simple, sécurisée et accessible sur un réseau local.

Ce sujet permet de mobiliser les compétences acquises en algorithmique, programmation orientée objet, développement web, bases de données, administration système et modélisation UML.

2. Objectifs pédagogiques

Le projet a pour objectifs de :

- Mettre en pratique les notions d'algorithmique et de programmation orientée objet.
- Développer une application web dynamique avec séparation entre frontend, backend et base de données.
- Exploiter un SGBD relationnel pour gérer les données des utilisateurs, catégories, budgets et transactions.
- Appliquer les notions de sécurité informatique de base : authentification, contrôle d'accès, validation des données.
- Déployer une application sur un serveur local accessible en réseau.
- Concevoir un système d'information à l'aide d'outils de modélisation adaptés.
- Produire une documentation technique claire et structurée.

3. Travail demandé

Les étudiants, en groupes de **2 à 3 personnes**, doivent concevoir, modéliser, développer, tester et déployer une application web permettant à plusieurs utilisateurs de :

Fonctionnalités principales

- Créer et gérer des **comptes utilisateurs sécurisés**
- Se connecter avec un compte utilisateur;
- Ajouter, modifier et supprimer des **revenus** et des **dépenses** ;
- définir des catégories de dépenses ;
- Organiser les transactions par **catégories** (logement, transport, loisirs...)
- Définir des budgets par catégorie ou par période(budgets mensuels ou hebdomadaires...);
- consulter l'état du budget ;
- collaborer sur un budget partagé (Partager un budget entre plusieurs utilisateurs (colocation, famille, groupe d'amis...) ;
- suivre les écarts entre dépenses prévues et réelles (**dépassement de budget avec alerte**);
- visualiser des statistiques des dépenses via un **tableau de bord graphique** ;
- gérer les rôles et les droits d'accès selon le profil utilisateur.

L'application doit être développée selon un **modèle de développement séquentiel** de type **cascade** ou **cycle en V**, en prenant en compte l'ensemble des étapes du processus logiciel.

4. Description générale de l'application

4.1 Idée générale

L'application permet à un ou plusieurs utilisateurs de gérer de manière collaborative leurs finances personnelles. Chaque utilisateur peut saisir ses opérations financières, organiser ses dépenses en catégories, définir des plafonds budgétaires et consulter des tableaux de bord visuels pour mieux comprendre son comportement financier.

L'aspect collaboratif permet à plusieurs membres de participer à un budget commun, par exemple :

- budget familial ;
- budget de colocation ;
- budget d'un groupe d'étudiants ;
- organisation des dépenses pour un événement.

4.2 Public cible

L'application s'adresse à :

- des étudiants ;
- des colocataires ;
- des familles ;
- des petits groupes ayant besoin de suivre des dépenses communes.

5. Besoins fonctionnels

5.1 Gestion des utilisateurs

L'application doit permettre :

- la création de comptes utilisateurs ;
- la connexion et la déconnexion sécurisées ;
- la modification du profil utilisateur ;
- la gestion des mots de passe ;
- l'attribution de rôles.

5.2 Gestion des rôles et des accès

Deux types d'utilisateurs minimum doivent être prévus :

Utilisateur

- créer compte, modifier profil et demander la suppression de compte ;
- gérer ses propres revenus et dépenses ;
- rejoindre ou créer un budget partagé ;
- consulter les statistiques liées aux budgets auxquels il participe.

.....

Administrateur

- valider la création d'un compte par l'envoi d'un email à la personne concernée ;
- Consulter comptes, valider demande de suppression de compte...
- gérer les rôles ;
- superviser l'ensemble du système ;
- consulter tous les budgets partagés si nécessaire.
- Consulter les statistiques globales.

.....

5.3 Gestion des revenus et dépenses

L'utilisateur doit pouvoir :

- ajouter une transaction (recette ou dépense);
- choisir son type : revenu ou dépense ;
- renseigner un montant ;
- indiquer une date ;
- ajouter une description ;
- affecter la transaction à une catégorie ;
- modifier ou supprimer une transaction.

5.4 Gestion des catégories

L'application doit proposer :

- des catégories par défaut : alimentation, transport, logement, santé, loisirs, études, etc.
- la possibilité d'ajouter des catégories personnalisées ;
- la modification ou suppression de catégories selon les droits de l'utilisateur.

5.5 Gestion des budgets

L'utilisateur doit pouvoir :

- créer un budget individuel ou partagé ;
- définir une période budgétaire : mensuelle, hebdomadaire ou personnalisée ;
- fixer un plafond global ;
- fixer des plafonds par catégorie ;
- suivre l'évolution du budget en temps réel.

5.6 Gestion collaborative

L'application doit permettre :

- la création d'un budget partagé ;
- l'ajout de membres à un budget ;
- la consultation commune des dépenses ;
- l'identification de l'auteur de chaque transaction ;
- éventuellement l'ajout de commentaires sur certaines opérations.

5.7 Alertes et suivi

Le système peut inclure :

- une alerte lorsque le budget atteint un certain seuil ;
- une alerte en cas de dépassement ;
- un indicateur d'état : budget maîtrisé, proche de la limite, dépassé.

5.8 Tableau de bord

Le tableau de bord doit présenter au minimum :

- le total des revenus ;
- le total des dépenses ;
- le solde disponible ;
- le pourcentage de budget consommé ;
- la répartition des dépenses par catégorie ;
- l'évolution des dépenses dans le temps.

Des représentations graphiques peuvent être utilisées :

- courbe d'évolution : Évolution des dépenses dans le temps ;
- camembert ; histogramme ;
- Suivi de la consommation budgétaire.
- Indicateurs simples (épargne, taux de dépenses, etc.)

6. Besoins non fonctionnels

6.1 Performance

- L'application doit offrir un temps de réponse raisonnable pour les opérations courantes.

6.2 Fiabilité

- Les données enregistrées doivent être cohérentes et non dupliquées.
- L'application doit éviter les erreurs critiques lors de l'utilisation normale.

6.3 Sécurité

- Authentification obligatoire.
- Protection des mots de passe par hachage.
- Contrôle des droits d'accès selon les rôles.
- Validation des entrées utilisateur.
- Protection contre les accès non autorisés.

6.4 Ergonomie

- L'interface doit être simple, claire et intuitive.
- La navigation doit être fluide.
- Les formulaires doivent être compréhensibles.

6.5 Maintenabilité

- Le code doit être structuré en modules.
- Les fonctions doivent être réutilisables.
- Le projet doit être documenté.

6.6 Portabilité

- L'application doit pouvoir être exécutée via un navigateur web standard.

7. Contraintes techniques

L'application devra être développée avec les technologies suivantes :

- **Frontend** : HTML, CSS, JavaScript...
- **Backend** : PHP (et éventuellement Java, Python)
- **Base de données** : MySQL ou Oracle

- **Système d'exploitation** : Linux conseillé
- **Déploiement** : serveur local accessible en réseau local

8. Méthodologie de développement

Le projet devra suivre un modèle séquentiel de type **cascade** ou **cycle en V**.

Étapes attendues

1. Analyse et Spécification des besoins

- analyse des besoins ;
- élaboration du cahier des charges fonctionnel ;
- identification des acteurs et fonctionnalités.
- Diagramme de **cas d'utilisation UML**
- Description textuelle des scénarios (Ex : élaboration d'un budget collaboratif...)
- Diagrammes de séquence système (Ex : Gestion d'une alerte de dépassement de budget...)

2. Conception

- modélisation UML ;
- conception de la base de données ;
- architecture logicielle.
- Exemples de diagrammes (Diagramme de **classes** ; Diagramme d'**activités** ; Diagramme de **séquence de conception** ; Diagramme de **composants** ; Diagramme de **déploiement**)

3. Implémentation

- développement frontend ;
- développement backend ;
- connexion à la base de données.

4. Vérification

- tests unitaires ;
- tests d'intégration ;
- tests des fonctionnalités principales.

5. Validation

- démonstration fonctionnelle ;
- vérification de la conformité aux besoins ;
- correction finale.

9. Architecture technique

9.1 Architecture générale

L'application peut être organisée selon une architecture en 3 couches :

- **Couche présentation** : HTML, CSS, JavaScript
- **Couche métier** : PHP
- **Couche données** : MySQL / Oracle

9.2 Fonctionnement global

1. L'utilisateur accède à l'interface web depuis son navigateur.
2. Les requêtes sont envoyées au serveur web.
3. Le backend PHP traite les demandes.
4. Les données sont stockées ou récupérées depuis la base.
5. Les résultats sont affichés sous forme de pages dynamiques ou de graphiques.

10. Interface utilisateur attendue

L'application doit comporter au minimum les pages suivantes :

- page d'accueil ;
- page d'inscription ;
- page de connexion ;
- tableau de bord principal ;
- page de gestion des transactions ;
- page de gestion des budgets ;
- page de gestion des catégories ;
- page d'administration ;
- page de profil utilisateur.
-

L'interface devra être :

- lisible ;
- responsive si possible ;
- cohérente visuellement ;
- simple à utiliser.

11. Livrables

Les livrables attendus sont :

1. Code source documenté

- code frontend ;
- code backend ;
- scripts SQL ;
- commentaires et structure claire.

2. Rapport de projet

Le rapport doit contenir :

- introduction ;
- analyse des besoins ;
- conception de la solution ;
- choix technologiques ;
- description de la base de données ;
- captures d'écran ;
- tests réalisés ;
- difficultés rencontrées ;
- conclusion.

3. Présentation orale avec démonstration fonctionnelle

12. Grille d'évaluation proposée

Critère	Description	Points
Fonctionnalités réalisées	Gestion des utilisateurs, transactions, budgets, collaboration, tableau de bord...	20
Processus de développement et modélisation	Respect du modèle de développement, UML, rigueur de conception	40
Qualité technique	Structuration du code, bonnes pratiques, qualité frontend/backend	30
Gestion de la base de données	Modélisation, intégrité, requêtes, pertinence du SGBD	20
Sécurité et accès réseau	Authentification, rôles, sécurité de base, déploiement local	20
Documentation et rapport	Clarté, organisation, justification des choix	20
Présentation orale et démonstration	Qualité de l'exposé, démonstration, réponses aux questions	20
Travail d'équipe et organisation	Répartition des tâches, coordination, respect des délais	10
Originalité et ergonomie	Design, facilité d'utilisation, fonctionnalités supplémentaires éventuelles.	20

13. Pistes d'amélioration possibles

Des fonctionnalités bonus peuvent être ajoutées pour enrichir le projet :

- export PDF ou CSV des transactions ;
- notifications automatiques ;
- comparaison de plusieurs mois ;
- objectifs d'épargne ;
- prévision simple des dépenses futures ;
- version mobile responsive ;
- mode sombre ;

.....

14. Conclusion

Ce projet de **gestion collaborative de budget personnel** mobilise à la fois des compétences en développement logiciel, bases de données, sécurité, réseau et modélisation. Il présente une application de gestion de budget utilisable, tout en laissant la place à des extensions possibles.